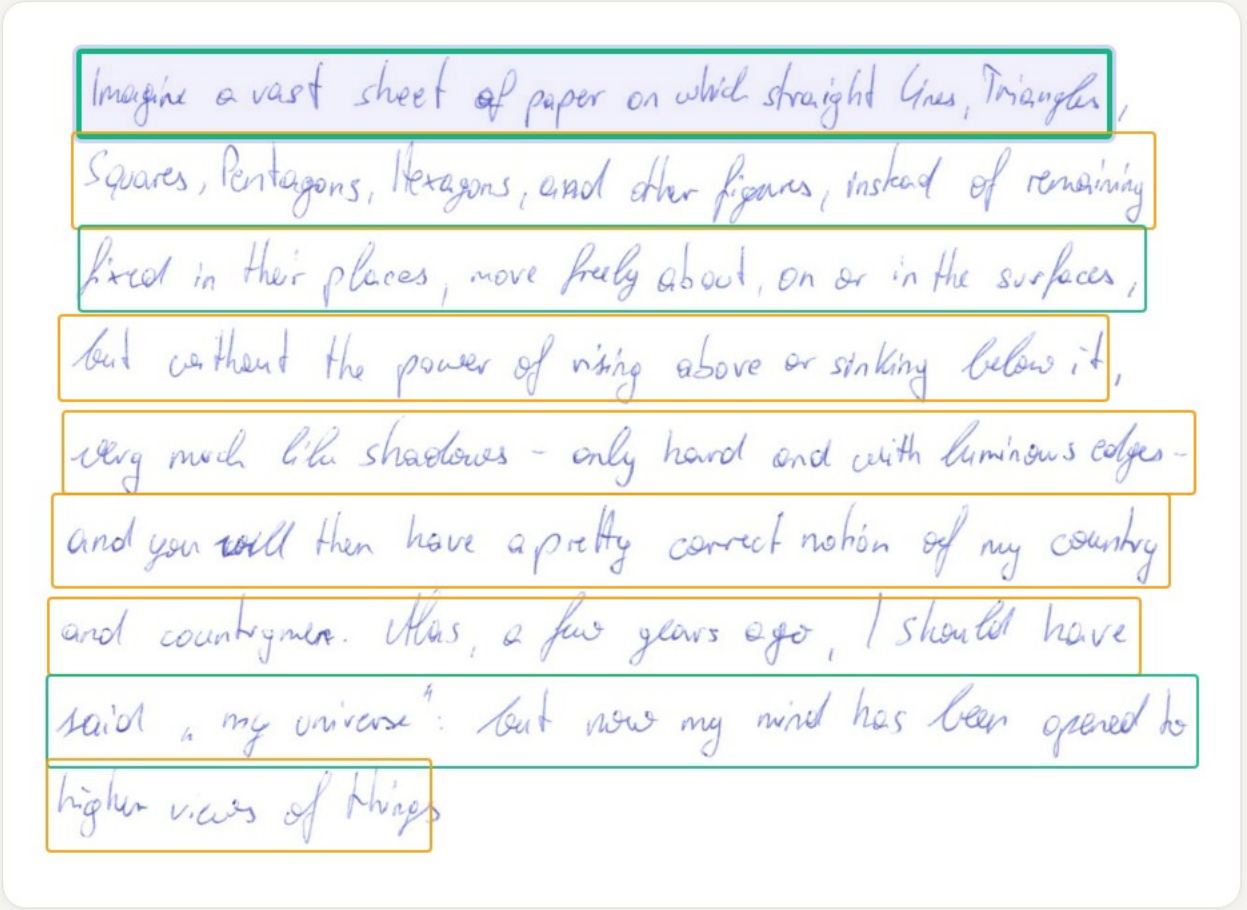
ADVANCED SOFTWARE ENGINEERING · 28/10/2026

Manuscript AI

A web app where historians **upload scans** of handwritten documents, get them **transcribed** by a model, and analyse the text.

Group 6 · Polina, Amaury, Ali, Fatou, Nicolas

University of Fribourg · Autumn 2026
Repository: `alocin98/ase-fribourg-history-scripts`



1 / 9 imagine a vast sheet of paper on which straight lines , Triangle 90%

Real output of our ML backend on `samples/english_page_test.jpg`

AGENDA

Five parts, five speakers

From what the app does today to what we will build in the second half.

1

Live demo

the working software and its idea

Ali · 2 min

2

Technology stack

model, backend, ML backend, frontend

Polina, Amaury, Ali, Fatou · 5 min

3

User stories

how we manage them, one example

Nicolas · 2 min

4

CI / CD

what happens when a branch is merged

Nicolas · 1.5 min

5

Outlook

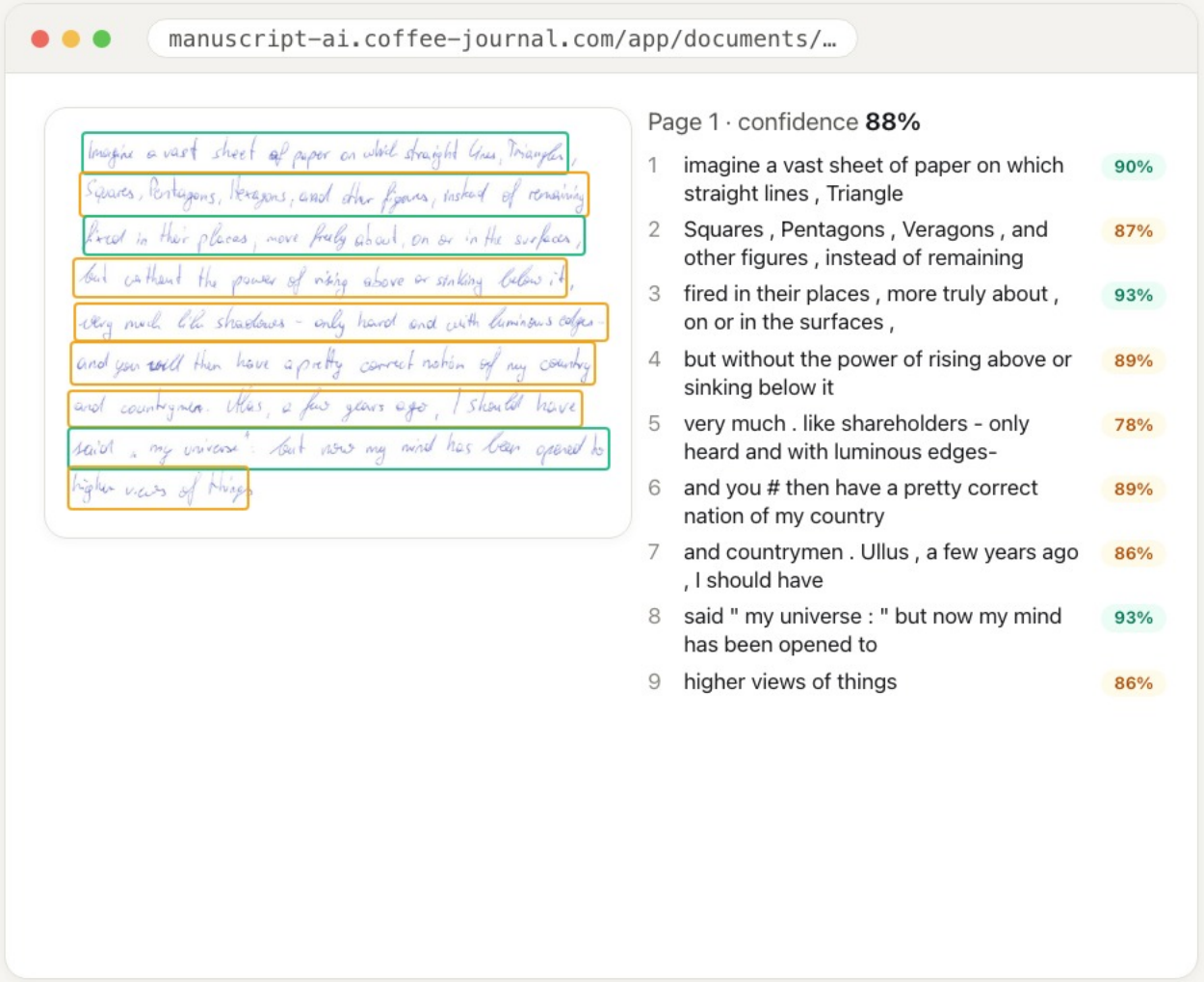
plans for the second half

Ali · 2 min

1 · LIVE DEMO — ALI

From a scan to a transcription

1. Create a **document** and upload a **ZIP** of page scans
   PNG / JPG pages, extracted and checked by the backend
2. **Preview** the pages of the document
3. Click **Transcribe**: one page or the whole document
   runs as a background job, the page shows the progress
4. Read the text **line by line**, with the model's **confidence**

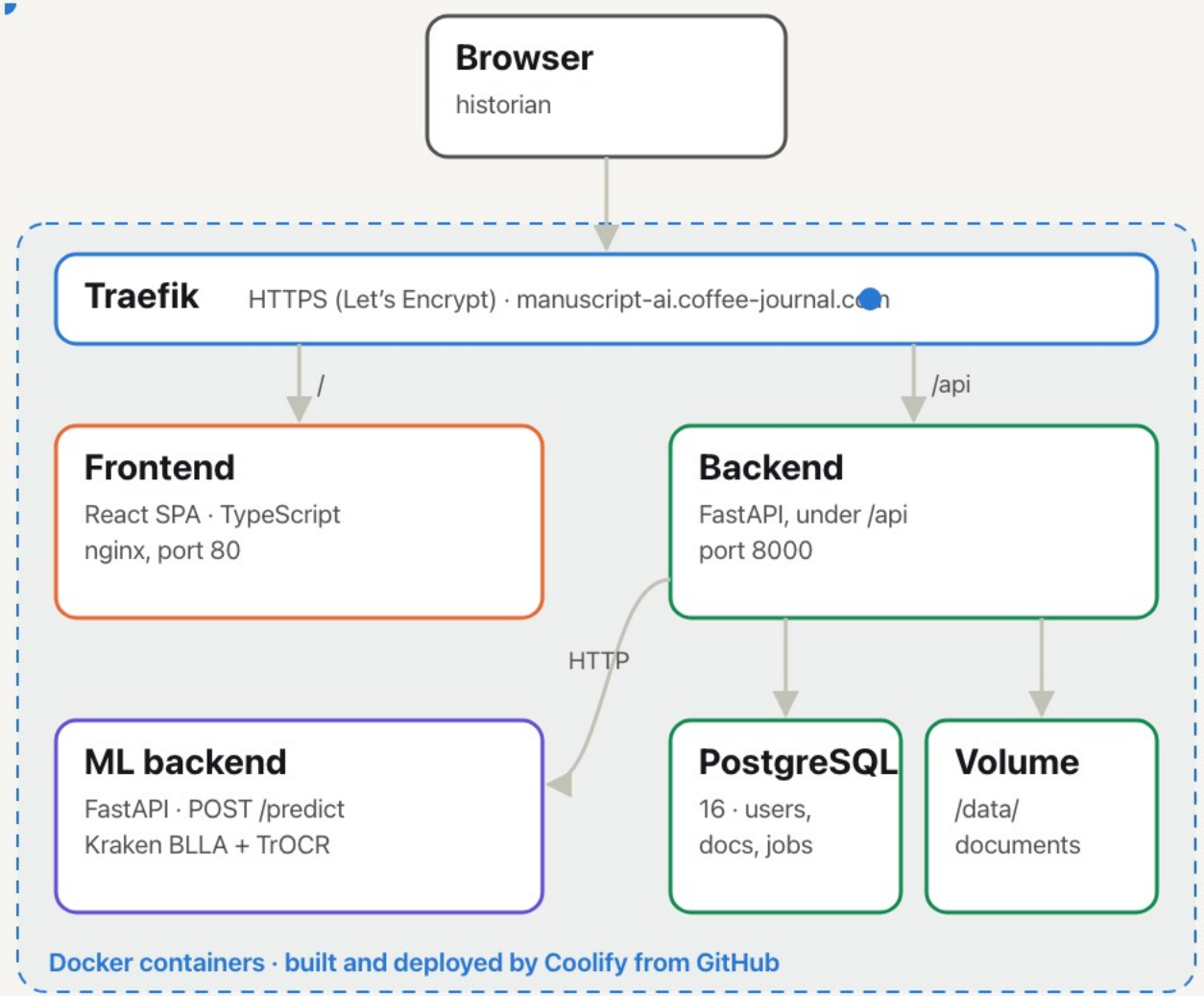


Schematic of the app (fallback if the live demo fails) · text and confidences = real model output

2 · TECHNOLOGY STACK — AMAURY

Four services

1. **Frontend**: React single-page app, served by nginx
2. **Backend**: FastAPI under `/api`, with PostgreSQL and a volume for the page images
3. **ML backend**: a separate FastAPI service that runs the model
   one endpoint, called only by the backend
4. Everything in **Docker**, deployed by **Coolify**; Traefik routes and adds HTTPS



2 · TECHNOLOGY STACK — POLINA

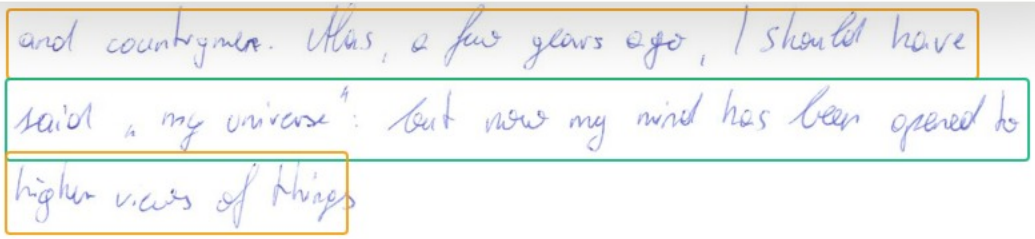
The model: find lines, then read them

1. Goal of the MVP: read a **whole handwritten page**
2. **Kraken BLLA** finds the text lines of the page
   pretrained default model; fragments on the same baseline are merged
3. **TrOCR** (`trocr-small-handwritten`) reads one line at a time
   Microsoft, fine-tuned on English handwriting (IAM)
4. Chosen and tested in a notebook, then moved into the ML backend
   `ml-models/english_cursive_mvp.ipynb` · story #18

BLLA: **19** fragments → **9** lines

TEXT OF THE WHOLE PAGE · TROCR

imagine a vast sheet of paper on which straight lines , Triangle
Squares , Pentagons , Veragons , and other figures , instead of remaining
fired in their places , more truly about , on or in the surfaces ,
but without the power of rising above or sinking below it
very much . like shareholders - only heard and with luminous edges-
and you # then have a pretty correct nation of my country
and countrymen . Ullus , a few years ago , I should have
said " my universe : " but now my mind has been opened to
higher views of things





line 1

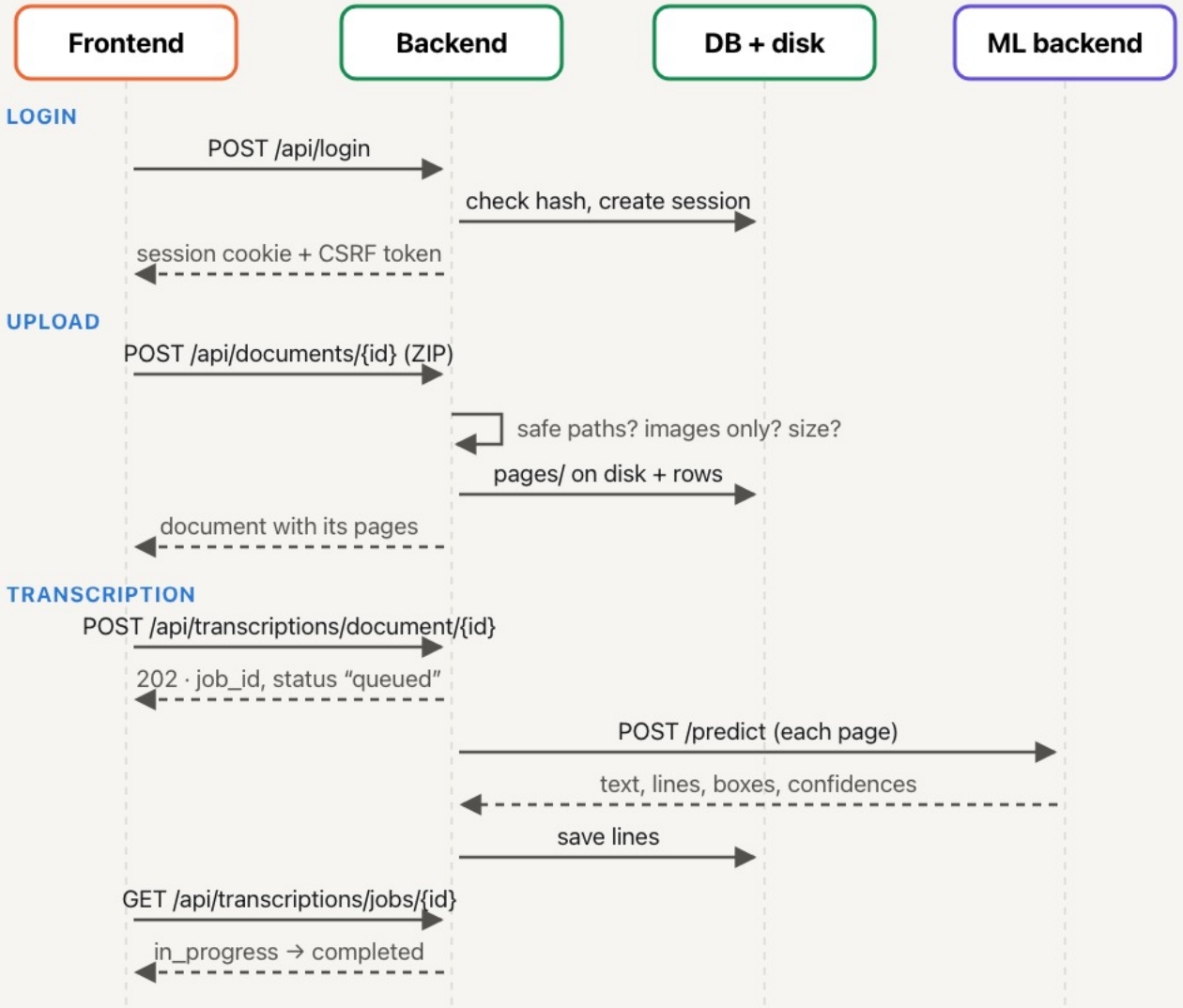
TrOCR → imagine a vast sheet of paper on which straight lines , Triangle

Real BLLA boxes and TrOCR text on `english_page_test.jpg`

2 · TECHNOLOGY STACK — AMAURY

The backend

1. **FastAPI**, **SQLAlchemy 2** (async) and **PostgreSQL 16**; Alembic migrations run at start
   routes → repositories → models
2. **Session** login: cookie + **CSRF** token, passwords hashed with Argon2
3. **Upload**: the ZIP is checked, the pages go to disk, their metadata to the database
4. **Transcription** runs as a job: `202` + job id at once, then the client polls the status



2 · TECHNOLOGY STACK — ALI

The ML backend

1. **One endpoint**: `POST /predict` with the page image
   models loaded once at start, weights baked into the image
2. It returns the text, and for each line its **box** in the image
3. … and a **confidence** per line and per word
   mean probability of their tokens
4. Making it run in Docker: pinned versions, memory limits

```
$ curl -F "file=@english_page_test.jpg" ml-backend/predict
# 1 worker · models loaded at start · 37.9 s for this page on CPU
```

`torch 2.4`

kraken 5.3 needs torch 2.4, but transformers won't load `.bin` weights with torch < 2.6 → TrOCR **safetensors** revision, pinned by commit

`MKLDNN=0`

with oneDNN, BLLA was killed (out of memory) in a 4 GB Docker VM; peak now ≈ 3.4 GB

`MAX_ASPECT 1.4`

wider images (e.g. a single line) are padded with white before BLLA; boxes stay in the original image

`offline image`

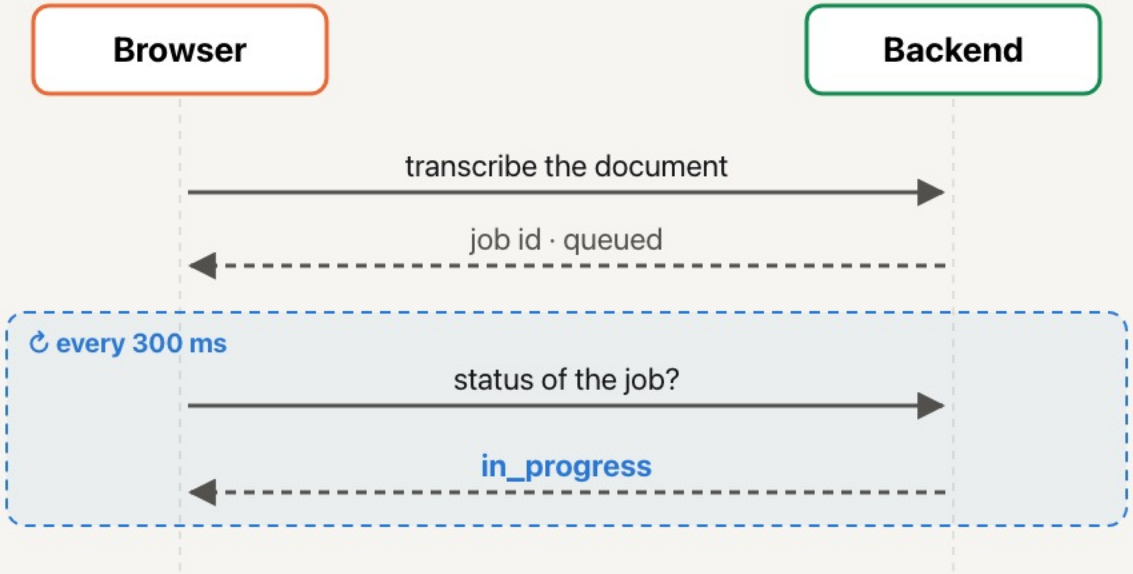
TrOCR weights are downloaded at build time; the container runs without network

Real response for `english_page_test.jpg` (shortened) · versions and memory: `webapp/ml-backend/readme.md`

2 · TECHNOLOGY STACK — NICOLAS / FATOU

The frontend

1. **React 19** + **TypeScript**, built by **Vite**, served as static files by **nginx**
   Tailwind · React Router · i18next (English / French) · Vitest
2. **Layers**: pages → auth context → API modules → one HTTP client
   every call goes to `/api` on the same domain: no CORS
3. **Transcription**: the frontend starts a **job**, then **polls** its status every 300 ms
   queued → in_progress → completed



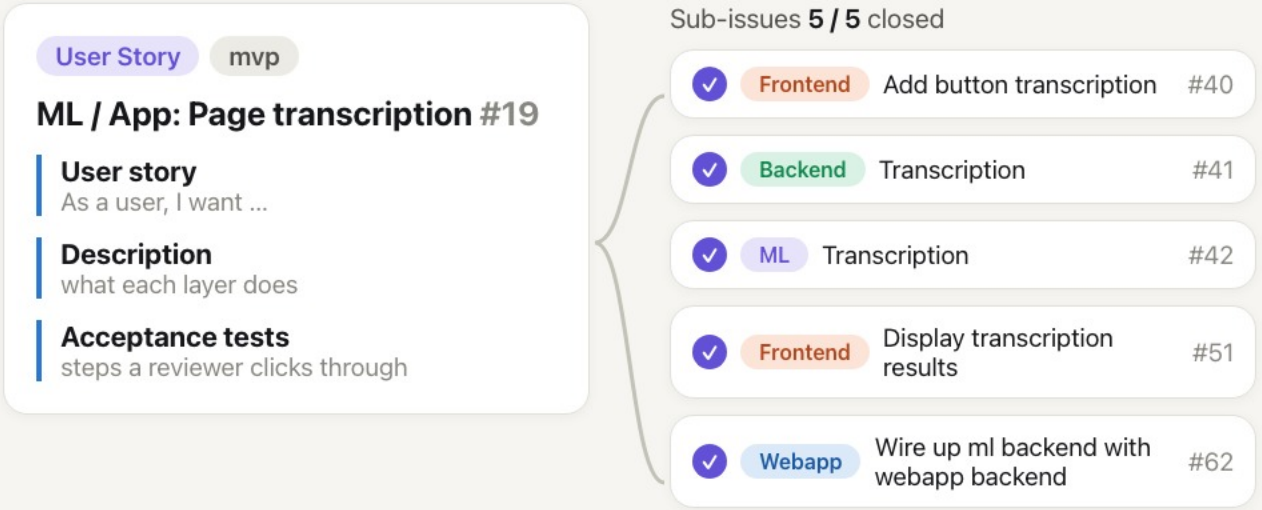
`webapp/frontend/src` : `api/client.ts` , `api/transcription.ts`

3 · USER STORIES — NICOLAS

How we manage user stories

1. A **GitHub issue** with the label `User Story`, written from the user's side
2. Always the same template: **story**, **description**, **acceptance tests**
3. **Sub-issues** are created by team members, because they know what to do within their part of the app

   Frontend · Backend · ML
4. Done when it meets our **Definition of Done**



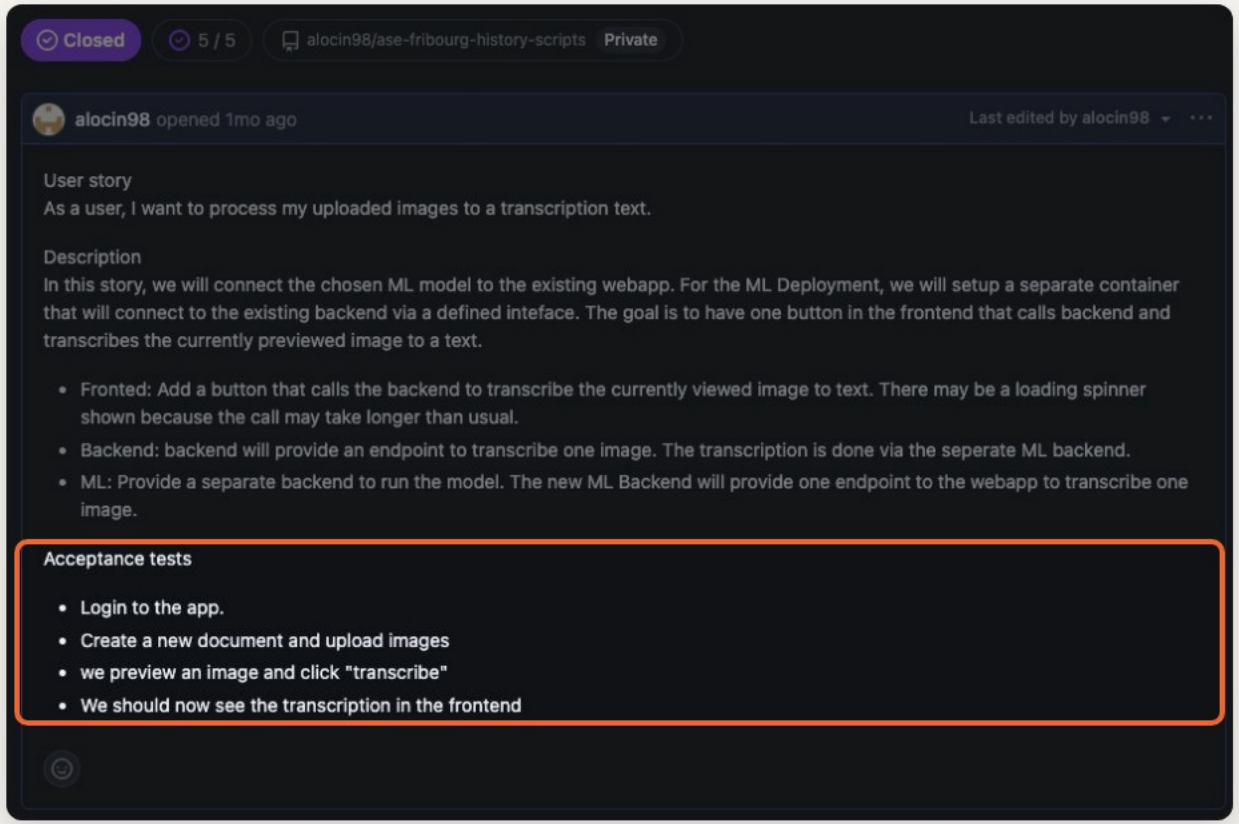
Definition of Done

- **Testing** — automated tests where it makes sense; current tests pass
- **Review** — by at least one other person, tested locally by the reviewer
- **Docs** — documentation, readme and agent files updated
- **Deployment** — deployed on dev and manually smoke tested

3 · USER STORIES — NICOLAS

Example: page transcription

1. **Story**: "As a user, I want to process my uploaded images to a transcription text."
2. **Description**: what the frontend, the backend and the ML part each have to do
3. **Acceptance tests**: the clicks a reviewer does to check it

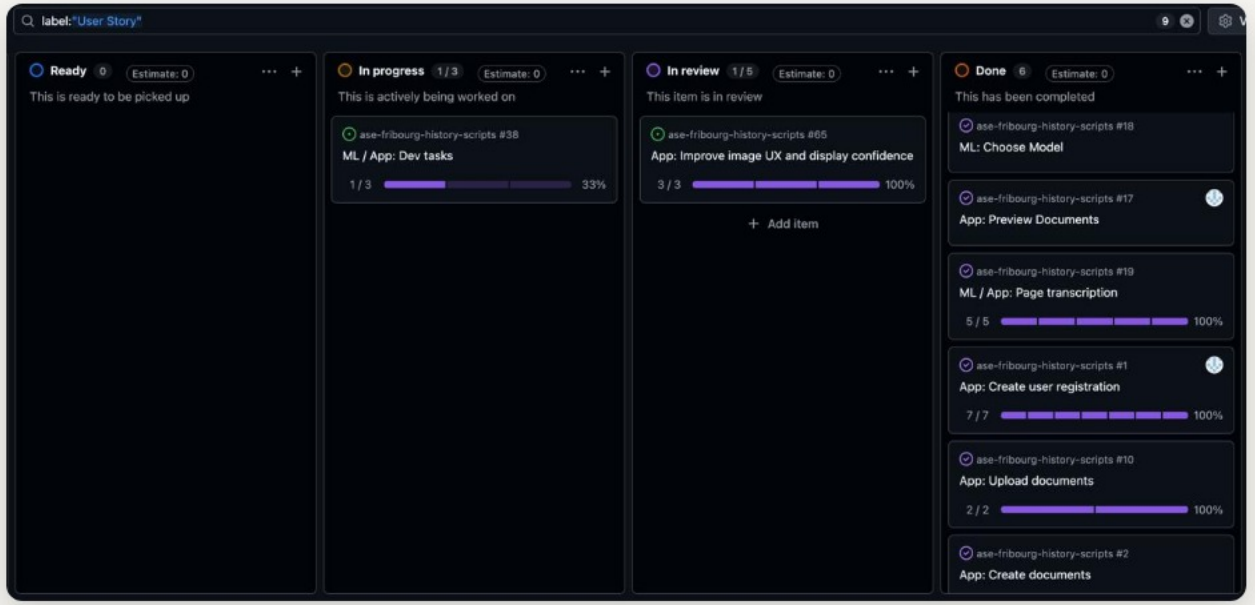


Issue #19 on GitHub

3 · USER STORIES — NICOLAS

Road to the MVP

1. We started from a **mockup** of the MVP
2. We track every story on a **GitHub project board**
   ready → in progress → in review → done
3. Released to production **step by step**



GitHub project board, filtered on label:"User Story"

dev → main: production deployments

7.10.2026
PR #32, #34

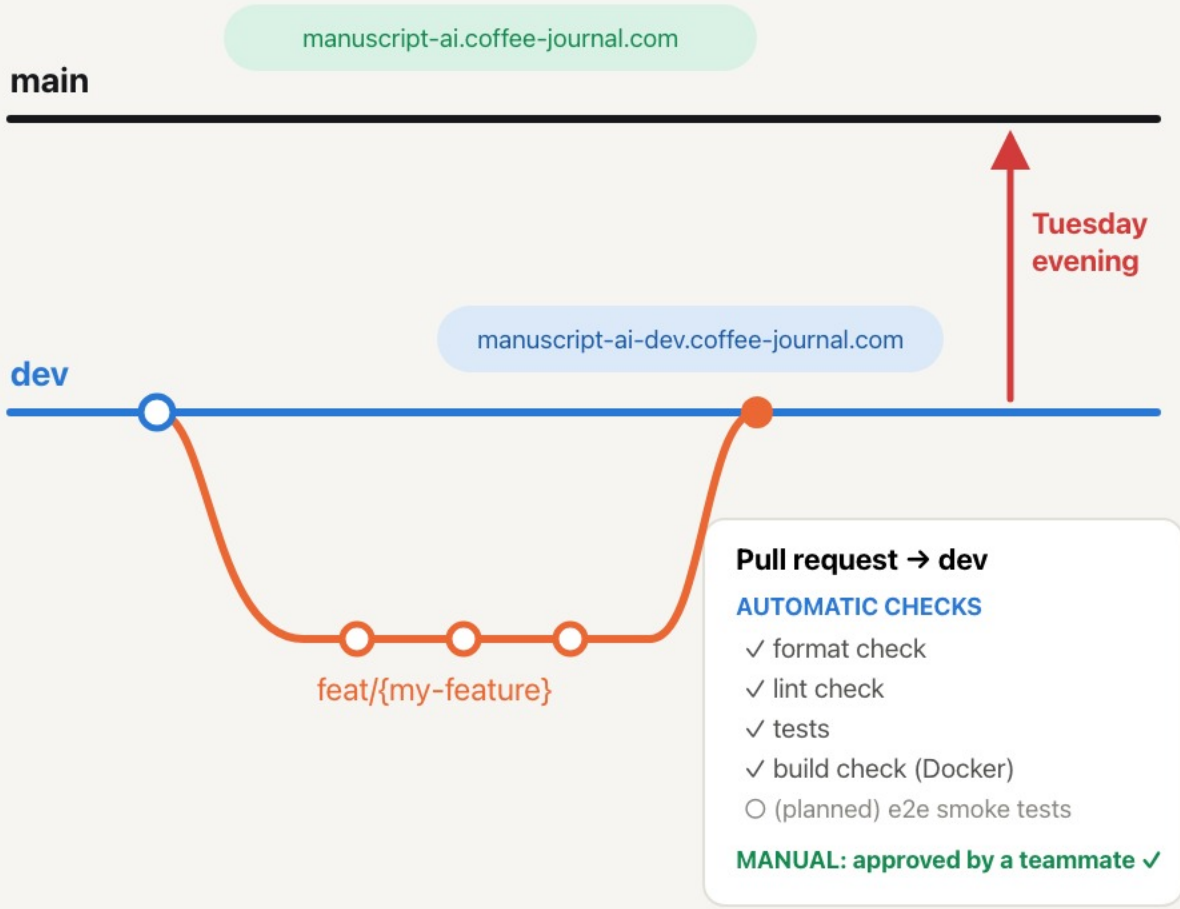
13.10.2026
PR #49

22.10.2026
PR #60

4 · CI / CD — NICOLAS

Git workflow

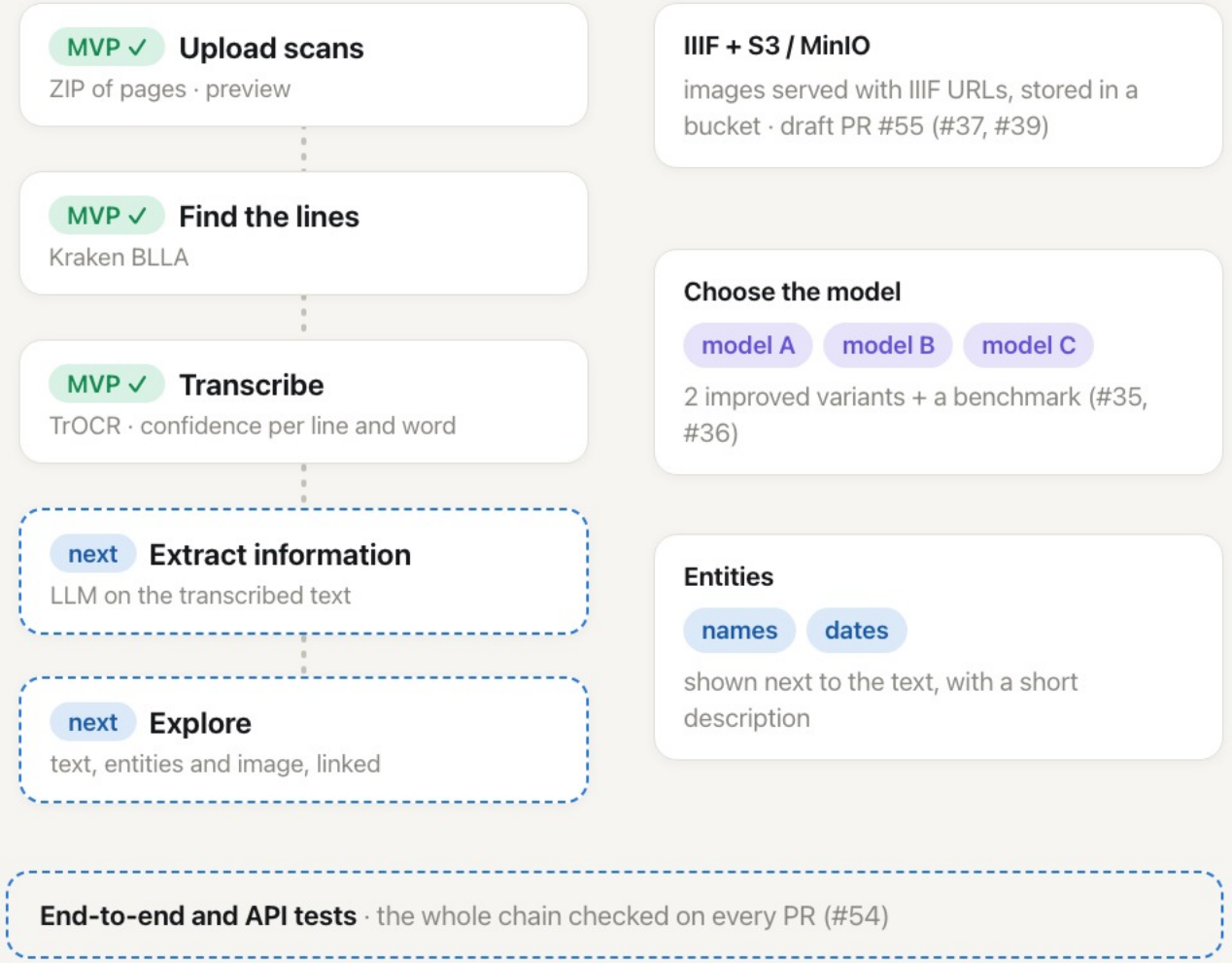
1. Work on a **feature branch** from `dev` : `feat/…` or `fix/…`
2. Open a **pull request**: automatic checks, and a review by a teammate
3. Merged into `dev` → deployed to the **dev** site
4. Each **Tuesday evening**, `dev` is merged into `main` → **production**



5 · OUTLOOK — POLINA

The second half

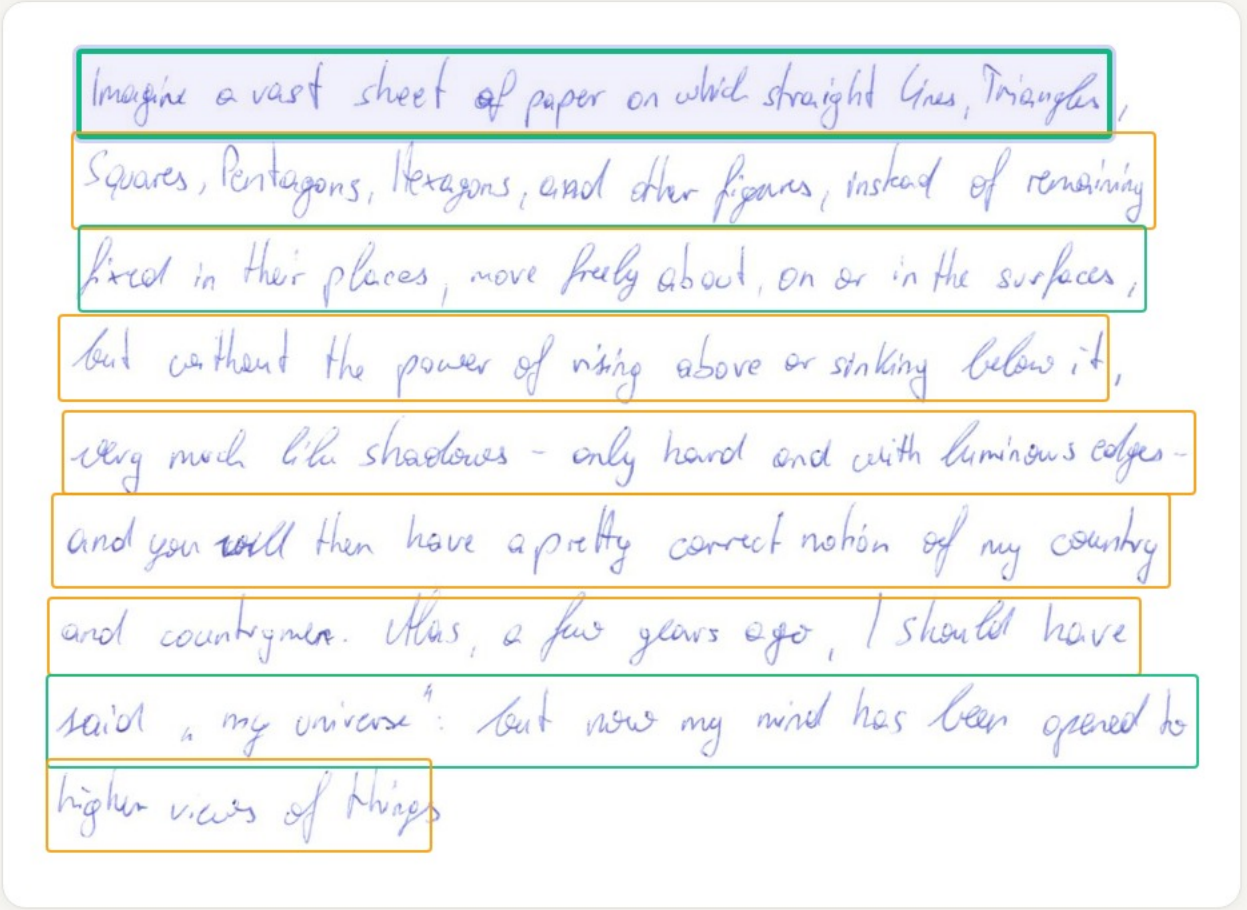
1. **Information extraction** with an LLM: names and dates, shown in the frontend
2. **2–3 models** the user can compare and choose from
   improve and benchmark the model (#36)
3. **IIIF**: open the images to standard viewers; storage on S3 / MinIO
   first draft: PR #55
4. **End-to-end** and API tests in the pipeline (#54)



Thanks

For listening to us. More to come later.

App: `manuscript-ai.coffee-journal.com`
Dev: `manuscript-ai-dev.coffee-journal.com`
Code: `alocin98/ase-fribourg-history-scripts`



1 / 9 imagine a vast sheet of paper on which straight lines , Triangle 90%